ensia The National School of Artificial Intelligence المدرسة الوطنية العليا للذكاء الاصطناعي

Introduction to Blockchain

Lab #1

Building a Simple Blockchain in Python

Objective:

In this lab, you will learn the foundational principles of blockchain technology by creating a simplified blockchain in Python. You will implement core blockchain functionalities such as block creation, mining, chaining, and validation. You will also deploy your blockchain as a basic web application using Flask.

Prerequisites:

- Python 3 installed on your system.
- Basic understanding of Python programming concepts.
- Familiarity with JSON data format.
- Basic understanding of web frameworks (Flask is helpful but not mandatory; basic concepts will be covered).

Instructions:

Step 1: Environment Setup

1. Open your preferred code editor (VSCode, PyCharm, etc.).
2. Install the necessary Flask package using pip:
   ```
   pip install flask
   ```
3. Create a new Python file named blockchain.py.

Step 2: Import Libraries

At the beginning of your blockchain.py file, import the following Python libraries. These will be essential for your blockchain implementation:

```
# Import necessary libraries
# ... (Your import statements will go here) ...
```

- **Directive:** Import libraries for handling:
  - Timestamps (for block creation).

        - SHA-256 hashing (for block fingerprinting and Proof-of-Work).
        - JSON format (for data serialization).
        - Creating a web application (using Flask).
    -

Step 3: Implement the Blockchain Class

1. Define a Python class named Blockchain. This class will encapsulate all the logic for your blockchain.
2. **`__init__` Method (Constructor):**

```
class Blockchain:

def __init__(self):
    # Initialize the blockchain
    # ... (Initialization code here) ...
    pass
```

    - **Directive:** Inside the __init__ method:
        - Initialize an empty list called chain to store the blocks of your blockchain.
        - Call the `create_block` method (which you will define next) to create the very first block of the blockchain (the "genesis block"). For the genesis block, use a proof of 1 and a previous_hash of '0'.
    -
3. **create_block Method:**

```
def create_block(self, proof, previous_hash):

# Create a new block and add it to the blockchain
# ... (Block creation logic here) ...
```

    - **Directive:** The create_block method should:
        - Accept proof and previous_hash as parameters.
        - Construct a block dictionary with the following keys:
            - 'index': The index of the block in the chain (should be the next index after the last block).
            - 'timestamp': The current timestamp.
            - 'proof': The proof value passed to the method.
            - 'previous_hash': The previous_hash value passed to the method.
        - Append the newly created block to the self.chain list.
        - Return the block that was created.

4. **print_previous_block Method:**

```
def print_previous_block(self):

# Return the last block in the blockchain
# ... (Code to return the previous block) ...
```

   - **Directive:** This method should simply return the last block added to the self.chain list. This will be useful for mining new blocks.

5. **proof_of_work Method:**

```
def proof_of_work(self, previous_proof):

# Implement the PoW algorithm to find a valid proof
# ... (Proof-of-Work logic here) ...
```

   - **Directive:** Implement a Proof-of-Work algorithm. A simple example is to:
     - Start with a new_proof value of 1.
     - Enter a loop that continues until a "valid proof" is found.
     - Inside the loop:
       - Create a hash of a combination of new_proof and previous_proof (e.g., using SHA-256).
       - Check if the beginning of the hash meets a certain "difficulty" criteria (e.g., starts with '00000').
       - If the criteria is met, a valid proof is found, and the loop should exit.
       - If not, increment new_proof and continue the loop.
     - Return the new_proof that satisfies the Proof-of-Work criteria.

6. **hash Method:**

```
def hash(self, block):

#Hash a block into a unique fingerprint
# ... (Hashing logic here) ...
```

- **Directive:** This method should:
  - Take a block (dictionary) as input.
  - Convert the block into a JSON string, ensuring keys are sorted for consistency.
  - Encode the JSON string into bytes.
  - Generate a SHA-256 hash of the encoded block.
  - Return the hexadecimal representation of the hash.

7. **chain_valid Method:**

```
def chain_valid(self, chain):

# Validate the entire blockchain for integrity
# ... (Blockchain validation logic here) ...
```

- **Directive:** Implement a blockchain validation method that:
  - Takes the chain (list of blocks) as input.
  - Starts by checking the genesis block.
  - Iterates through the blockchain, block by block, starting from the second block:
    - For each block, verify:
      - That the previous_hash in the current block matches the hash of the *previous* block.
      - That the proof of the current block is valid according to your Proof-of-Work algorithm (re-calculate the hash and check the difficulty criteria).
    - If any validation check fails for any block, the blockchain is invalid, and the method should return False.
  - If all blocks are valid, the blockchain is valid, and the method should return True.

Step 4: Set Up the Flask Web Application

Initialize a Flask application instance and instantiate your Blockchain class:

```
    # Initialize Flask and Blockchain
# ... (Flask and Blockchain initialization code) ...
```

- **Directive:** Create a Flask application instance named app.
- Create an instance of your Blockchain class named blockchain.

2. **Define the /mine_block Route:**

```
# Define Flask route to mine a new block

# ... (Route definition for /mine_block) ...
def mine_block():
# Logic to mine a new block
# ... (Mining logic for the route handler) ...
```

   - **Directive:** Create a Flask route using the @app.route decorator for the URL /mine_block that accepts GET requests.
   - Inside the mine_block function:
     - Get the previous_block from your blockchain.
     - Get the previous_proof from the previous_block.
     - Run the proof_of_work algorithm to find a proof for the new block, using the previous_proof.
     - Get the previous_hash of the previous_block.
     - Create a new block using blockchain.create_block with the calculated proof and previous_hash.
     - Construct a JSON response containing:
       - A message indicating a block was mined.
       - The index of the new block.
       - The timestamp of the new block.
       - The proof of the new block.
       - The previous_hash of the new block.
     - Return the JSON response with an HTTP status code of 200 (OK).

Define the /get_chain Route:

```
# Define Flask route to display the entire blockchain
# ... (Route definition for /get_chain) …
def display_chain():
# Logic to display the blockchain
# ... (Display chain logic for the route handler) ...
```

   - **Directive:** Create a Flask route for /get_chain that accepts GET requests.
   - Inside the display_chain function:
     - Construct a JSON response containing:
       - The complete chain from your blockchain.
       - The length of the chain.
     - Return the JSON response with an HTTP status code of 200 (OK).

3. **Define the /valid Route:**

```
# Define Flask route to validate the blockchain

# ... (Route definition for /valid) …
def valid():
# Logic to validate the blockchain
# ... (Validation logic for the route handler) ...
```

   - **Directive:** Create a Flask route for /valid that accepts GET requests.
   - Inside the valid function:
     - Call the blockchain.chain_valid method to validate your blockchain.
     - Based on the validation result (True or False), create a JSON response with a message indicating whether the blockchain is valid or not.
     - Return the JSON response with an HTTP status code of 200 (OK).

Step 5: Run the Flask Application

1. Add the following lines at the end of your blockchain.py file to run the Flask development server:

```
# Run the Flask application
# ... (Flask app run code) ...
```

   - **Directive:** Use app.run() to start the Flask development server. Configure it to run on host '127.0.0.1' and port 5000.
2. Save your blockchain.py file. And run your Python script from your terminal:

```
python blockchain.py
```

Step 6: Interact with Your Blockchain Web Application

1. **Mine a Block:** Open a web browser and go to http://127.0.0.1:5000/mine_block. Observe the JSON output representing the mined block.
2. **Display the Blockchain:** Navigate to http://127.0.0.1:5000/get_chain. Examine the JSON output showing the entire blockchain structure.
3. **Validate the Blockchain:** Access http://127.0.0.1:5000/valid. Check the JSON response to see if your blockchain is reported as valid.

Step 7: Understanding the Outputs

Try to understand the outputs of calling the previous functions

Bonus Challenge:

Define the Blockchain Class (with Difficulty Parameter)

Implement the Blockchain class as described before, but now we will add a mechanism to control the mining difficulty.

`__init__` Method (Constructor) - Add Difficulty:

```
 class Blockchain:

def __init__(self, difficulty=4):  # Default difficulty (4)

self.chain = []

self.difficulty = difficulty  # Store the difficulty level

 self.create_block(proof=1, previous_hash='0')
```

- **Directive:** Modify the __init__ method to:
  - Accept an optional parameter difficulty with a default value (e.g., 4). This parameter will control the mining difficulty.
  - Store this difficulty value as an attribute of the Blockchain class (e.g., self.difficulty).
- Update also the proof of work method.